

FIT5221 - Assignment 2

This assignment focuses on concepts and practical skills in building and training image classifiers with Convolutional Neural Networks (CNNs). You are expected to demonstrate a comprehensive understanding of CNNs and ultimately put them into practice by constructing models for an image classification task.

The objective of this assignment is to build an image classification system with CNNs. There are 4 tasks:

- Task 1 requires you to build a simple CNN model for image classification, which serves as a baseline for the next tasks.
- Task 2 requires you to improve the baseline model from Task 1 by considering specific techniques/modifications.
- Task 3 requires you to combine the techniques you used in Task 2 to find the best (possible) model.
- Task 4 asks you to explore and implement other approaches to improve the performance of your best model in Task 3.

Available: **Wednesday, April 15, 2026.**

Submission due: **11.55 PM, Wednesday, May 6, 2026.**

Instructions

1. All code should be in Python ($\geq 3.10.x$). You should write appropriate comments to explain your implementations.
2. Build CNN models using **PyTorch**.
3. Late submission penalty: 5% of the total marks per day or part thereof.
4. All submissions must be made through Moodle only.
5. Plagiarism cases will be dealt with following Monash policy.
6. AI & Generative AI tools **MUST NOT BE USED** in this assignment.
7. **Please read the Instructions and Requirements for each task carefully.**

Submission

Your submission contains two files:

1. **A ZIP file** named "A2_<YourMonashID>.zip" (e.g. A2_12345678.zip), including:

- **A Jupyter notebook** named `StudentName\_ID.ipynb`, that presents all your implementations, inputs, and outputs for each task. Please rename the Jupyter notebook accordingly. In the first cell of your notebook, you must specify your full name and student ID. The Jupyter notebook should include comments that explain your code.
- **All required model checkpoint files.**
- Any extra files or folders that are required to run your notebook and reproduce your results.

2. **A separate report in PDF format** named `Report\_StudentName\_ID.pdf`, that describes your approaches. Please rename the report file accordingly. You must specify your full name and student ID at the beginning of the report. **NOTE: this report must be a separate document. Do not submit the PDF export of your Jupyter notebook. For each task, your report may include (but not limited to):**

- Discussion of your implementation and parameter choices.
- Discussion of your results.
- Any issues you encountered, comparisons, or analysis (if applicable).
- **Any additional information required by the task instructions/requirements.**

NOTES:

- **Ensure that the notebook runs without errors. We will not modify your code. When run with the provided checkpoints, your code should reproduce approximately the same results as reported. We will re-run your notebook, using your provided checkpoints on our computers.**
- **For all tasks, you are free to tune training hyperparameters (e.g., learning rate, optimizer, scheduler, batch size, and number of training epochs)**
- **Ensure the models' performance is printed out in your notebook as well as included in your report.**
- **The inference time for both Task 1 and Task 3 should be less than 30 minutes using the Google Colab's GPUs. Marks might be deducted if your code takes more than 30 minutes for inference.**
- **The task descriptions are elaborated in the following pages.**

Task 1 - Baseline CNN model (4 marks)

In this task, you will build an image classification system using a pre-defined CNN architecture. The system will be trained and tested using the Tiny ImageNet-60k dataset. This dataset is a subset of Tiny ImageNet (<https://huggingface.co/datasets/zh-plus/tiny-imagenet>), consisting of 200 classes with a total of 60,000 training images and 10,000 test images. **The dataset will be used in all tasks. We provide simple scripts to load the datasets and create dataloaders in the notebook.**

Layer	Hyperparameters	Output Shape	Param #
Input	/	(3, 64, 64)	0
Conv2D	# Kernels: 16; Kernel size: 7x7 Stride: 1; Padding: 0 Activation: ReLU	(16, 58, 58)	2,368
MaxPool2D	Kernel size: 2; Stride: 2	(16, 29, 29)	0
Conv2D	# Kernels: 32; Kernel size: 5x5 Stride: 1; Padding: 2 Activation: ReLU	(32, 29, 29)	12,832
MaxPool2D	Kernel size: 2; Stride: 2	(32, 14, 14)	0
Conv2D	# Kernels: 32; Kernel size: 5x5 Stride: 1; Padding: 2 Activation: ReLU	(32, 14, 14) (Note: this is the final feature maps)	25,632
MaxPool2D	Kernel size: 2; Stride: 2	(32, 7, 7)	0
Flatten	/	1568	0
Linear	Neuron: 196, with Bias Activation: ReLU	196	307,524
Linear	Neuron: 200, with Bias	200	39,400
Total number of parameters			387,756

Instructions

- Design your baseline model based on the above descriptions **using PyTorch**.
- Using torchinfo, **print out the summary** of this model.
- **Train and evaluate** the baseline model on the Tiny ImageNet-60k dataset.
- **Plot the top-1 accuracy and loss for both the training and test sets** across epochs, and report the best test set accuracy.

Hints

- The teaching team's baseline models achieved approximately 20% Top-1 accuracy on the test set. This figure is for reference only; you are not expected to exactly reproduce it.
- Consider saving your best model checkpoints based on test loss or test accuracy for use in later tasks.

Requirements: Your notebook must include the implementation code, model summaries, **plots showing top-1 accuracy and loss for both training and testing sets across epochs**, and evaluation results on the test set.

Notes: You are not required to submit model checkpoints for Task 1. You may skip the report for this task.

Task 2. Improve the baseline model from Task 1 (7 marks)

You will need to improve the baseline model in Task 1 by considering the following techniques/modifications:

- Data augmentation (**1 mark**)
- Incorporating normalization layers and dropout layers (**1 mark**)
- Modify convolution kernel sizes and the number of convolution filters (**1 mark**)
- Add more layers to increase depth and experiment with different pooling strategies for the final feature maps before feeding them into the linear layers. (**1 mark**)
- Using residual blocks (**1.5 marks**)
- Using inception modules (**1.5 marks**)

Instructions

For each sub-task above, you must:

- **Apply the technique to the baseline model from Task 1.** Experiment with multiple sets of hyperparameters (e.g., varying the number of layers, their positions, kernel sizes, or combinations of data augmentation techniques). For example, when using normalization layers, experiment with different numbers of normalization layers and their positions within the network, then evaluate and report how these changes affect performance.
- **Plot the top-1 accuracy and loss for both the training and test sets across epochs, and report the best test set accuracy** for each variant of the model.
- **Report:** Provide observations and test accuracy. Explain why the approach should (or should not) improve upon the baseline model's performance.

Requirements

- Your notebook must include the implementation code, model summaries, **plots showing top-1 accuracy and loss for both training and testing sets across epochs**, and evaluation results on the test set.
- To ensure a clean submission, **remove excessive training logs or print statements. Only essential outputs (such as final metrics and plots) should remain visible in the notebook. We may deduct your marks if your notebook contains excessive logs.**

Notes: You are not required to submit model checkpoints for Task 2.

Task 3. Find the best model (5 marks)

In this task, you will **combine ONLY the techniques and architectural modifications you implemented in Task 2** to construct the best possible model. You are encouraged to experiment with various combinations of these techniques to maximize test accuracy.

Instructions

- **Implementations (1 mark)**
 - **Submit the implementation of your best model along with the model checkpoint.** Your notebook **MUST** include a code cell that instantiates your best model architecture and loads the weights from a checkpoint path.
 - **Plot the top-1 accuracies and losses for both the training and testing sets across epochs and report the best test set accuracy.**
- **Report (1 mark)**
 - Provide descriptions and test accuracy of your best model. Justify your choice of architectural combinations. Discuss why you selected these specific techniques.
- Based on the Top-1 accuracy of your best model on the test set, marks will be awarded as follows:

Test accuracy	Marks awarded
Accuracy $\geq$ 45%	3 marks
45% > Accuracy $\geq$ 30%	2 marks
30% > Accuracy $\geq$ 20%	1 mark
20% > Accuracy	0 marks

Requirements

- **You MUST train your model from scratch.** The use of pre-trained weights (e.g., from ImageNet) is strictly prohibited.
- **You MUST NOT use pre-defined backbone architectures from libraries** (e.g., torchvision, timm, etc.). Your model **MUST BE** built manually using PyTorch layers, **incorporating only the techniques explored in Task 2.**
- **The total number of parameters in your best model MUST NOT exceed 40 million.** Marks will be deducted if this limit is exceeded.
- Your notebook must include the implementation code, model summaries, **plots showing top-1 accuracy and loss for both training and testing sets across epochs**, and evaluation results on the test set.
- **You MUST submit your model checkpoints.** Failure to provide valid checkpoints that reproduce your reported results will result in zero marks for this task.

Task 4 - Explore image classification (4 marks)

In this task, you are required to explore and implement additional approaches beyond those used in Task 2 and Task 3 to further improve test accuracy. These approaches can be derived from research papers or your own novel ideas.

If the approaches you use in this task achieve **better accuracy than your best model in Task 3**, you will receive one mark. **(1 mark)**

Instructions

- **Implementations (1.5 marks)**
 - **Submit the implementation of your proposed model along with the model checkpoint.** Your notebook **MUST** include a code cell that instantiates your proposed model architecture and loads the weights from a checkpoint path.
 - **Plot the top-1 accuracies and losses for both the training and testing sets across epochs and report the best test set accuracy.**
- **Report (1.5 marks)**
 - For each approach, provide a clear description (e.g., how it works and why it improves performance) along with its test accuracy.
 - Citations/references for any methods adopted from existing literature.

Requirements

- Your notebook must include the implementation code, model summaries, **plots showing top-1 accuracy and loss for both training and testing sets across epochs**, and evaluation results on the test set.
- **You MUST submit your model checkpoints.** Failure to provide valid checkpoints that reproduce your reported results will result in zero marks for this task.
- You **SHOULD NOT** reuse techniques/modifications from Task 2 with a different set of hyperparameters and claim them as new approaches for this task.

===== The end =====